

Assignment 2: Training neural network: Learning rate, dropout, activation function

Trong bài thực hành này, chúng ta sẽ tìm hiểu các vấn đề về learning rate, dropout, activation function thông qua bài toán phân loại chữ số viết tay trên bộ dữ liệu MNIST.

```
In [ ]: import warnings
warnings.filterwarnings('ignore')

import numpy as np
import matplotlib.pyplot as plt
import cv2
import torch
import os
from torch import nn
import torch.nn.functional as F
from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor

from torch.utils.tensorboard import SummaryWriter

%load_ext tensorboard

use_cuda = torch.cuda.is_available()
```

Phần 1: Quan sát dữ liệu

MNIST là tập dữ liệu ảnh đen trắng các chữ số viết tay, có cùng kích thước 28x28. Chữ số trong ảnh đã được căn chỉnh vào tâm. Đây là tập dữ liệu rất phù hợp cho việc thử nghiệm các kỹ thuật huấn luyện và nhận dạng mẫu mà không đòi hỏi quá nhiều công sức tiền xử lý.

Bộ dữ liệu MNIST được chia sẵn thành 2 phần: tập dữ liệu huấn luyện gồm 60.000 ảnh, tập dữ liệu kiểm thử gồm 10.000 ảnh. Các ảnh trong bộ dữ liệu thuộc về một trong 10 lớp: 0, 1, 2,..., 9.

Thư viện PyTorch đã cung cấp sẵn một module để tải về MNIST:

```
In [2]: train_data = datasets.MNIST(
    root = 'data',
    train = True,
    download = True
)
test_data = datasets.MNIST(
    root = 'data',
    train = False
```

```

)
(x_train, y_train) = train_data.data[:].detach().numpy(), train_data.targets[:].det
(x_test, y_test) = test_data.data[:].detach().numpy(), test_data.targets[:].detach(
print('Training image: ', x_train.shape)
print('Testing image: ', x_test.shape)
print('Training label: ', y_train.shape)
print('Testing label: ', y_test.shape)

```

Training image: (60000, 28, 28)

Testing image: (10000, 28, 28)

Training label: (60000,)

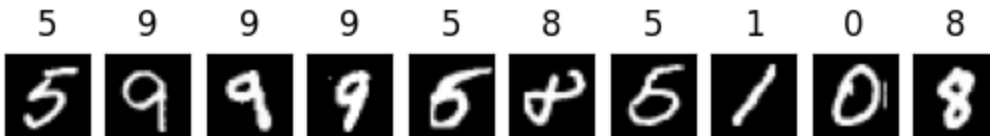
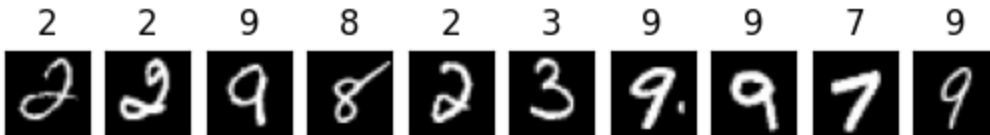
Testing label: (10000,)

Để dễ hình dung về dữ liệu, có thể sử dụng thư viện matplotlib quan sát một vài mẫu dữ liệu:

```

In [3]: for i in range(30):
        idx = np.random.randint(0, x_train.shape[0])
        image = x_train[idx]
        plt.subplot(3, 10, i + 1), plt.imshow(image, cmap='gray')
        plt.title(y_train[idx]), plt.xticks([]), plt.yticks([])
        plt.show()

```



```

In [4]: batch_size = 64
        num_classes = 10
        epochs = 20
        # input image dimensions
        img_rows, img_cols = 28, 28

        # PyTorch expects input as NCHW (Batch, Channels, Height, Width)
        # Reshape the input to NCHW format.

        x_train = x_train.reshape(x_train.shape[0], 1, img_rows, img_cols)

```

```
x_test = x_test.reshape(x_test.shape[0], 1, img_rows, img_cols)
input_shape = (1, img_rows, img_cols)
```

Trước khi đưa vào mô hình, cần chuẩn hoá giá trị điểm ảnh về trong khoảng [0,1] nhằm giúp các thuật toán tối ưu hội tụ nhanh hơn:

```
In [5]: x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255
```

```
In [6]: print(x_train.shape, y_train.shape)

(60000, 1, 28, 28) (60000,)
```

Phần 2: Xây dựng mô hình phân loại

Trong phần này, chúng ta sẽ xây dựng một mô hình phân loại đơn giản cho bài toán nhận diện chữ số viết tay sử dụng thư viện keras. Từ mô hình này ta sẽ thử nghiệm tác động của các yếu tố như learning rate, dropout, activation function đến quá trình huấn luyện mạng.

```
In [7]: class DeepModel(nn.Module):
    def __init__(self, dropout_rate):
        super(DeepModel, self).__init__()
        self.conv_1 = nn.Conv2d(1, 32, kernel_size = 3, stride = (1, 1), padding=0,
        self.conv_2 = nn.Conv2d(32, 64, kernel_size = 3, stride = (1, 1), padding=0

        self.dropout = nn.Dropout(dropout_rate)

        self.dense_1 = nn.Linear(12*12*64, 512)
        self.dense_2 = nn.Linear(512, 10)

    def forward(self, x):
        x = self.conv_1(x)
        x = nn.ReLU(inplace=True)(x)
        x = self.conv_2(x)
        x = nn.ReLU(inplace=True)(x)
        x = nn.MaxPool2d(kernel_size=2)(x)

        x = x.reshape(x.size(0), -1)
        x = self.dense_1(x)
        x = nn.ReLU(inplace=True)(x)
        x = self.dropout(x)

        output = F.log_softmax(self.dense_2(x))

        return output
```

Kiểm tra số lượng tham số

```
In [8]: simple_model = DeepModel(dropout_rate = 0.5)
if use_cuda:
```

```

    simple_model = simple_model.cuda()
for param in simple_model.parameters():
    if param.requires_grad:
        print('param autograd')
        break

input = torch.rand(2, 1, 28, 28)
if use_cuda:
    input = input.cuda()
output = simple_model(input) # type: torch.Tensor

model_parameters = filter(lambda p: p.requires_grad, simple_model.parameters())
params = sum([np.prod(p.size()) for p in model_parameters])
print('Number of parameter:', params)

```

param autograd
Number of parameter: 4742954

Khởi tạo Generator

```

In [9]: class Generator(Dataset):
        def __init__(self, images, labels):
            self.images = images
            self.labels = labels

        def __len__(self):
            return self.images.shape[0]

        def __getitem__(self, idx):
            return self.images[idx], self.labels[idx]

```

```

In [10]: training_data = Generator(x_train, y_train)
         train_dataloader = DataLoader(training_data, batch_size=32, shuffle=True)

```

```

In [11]: test_data = Generator(x_test, y_test)
         test_dataloader = DataLoader(test_data, batch_size=32, shuffle=True)

```

Phần 2.1: Vai trò của Dropout

Kỹ thuật dropout tắt đi một số kết nối một cách ngẫu nhiên trong mỗi lượt huấn luyện, giúp tránh hiện tượng đồng thích nghi (co-adaptation). Điều đó giúp mô hình bị quá khớp. Việc quá khớp thể hiện ở việc lỗi huấn luyện là rất nhỏ nhưng lỗi kiểm thử lại lớn. Phần thực hành tiếp theo nhằm minh họa cho tác dụng của kỹ thuật này.

- Dropout rate = 0

```

In [12]: use_cuda = torch.cuda.is_available() #GPU cuda
         best_loss = float('inf')

         model = DeepModel(dropout_rate = 0)

```

```
optimizer = torch.optim.Adam(model.parameters())
if use_cuda:
    model = torch.nn.parallel.DataParallel(model.cuda()) # , device_ids=[0, 1, 2,
    torch.backends.cudnn.benchmark = True
```

```
In [13]: def train(model, optimizer, epoch, writer):
print('\n ##### Train phase, Epoch: {} #####')
model.train()
train_loss = 0
running_loss = 0
print('\n Learning rate at this epoch is: ', optimizer.param_groups[0]['lr'], '\n')
for (batch_idx, target_tuple) in enumerate(train_dataloader):
    images, labels = target_tuple
    if use_cuda:
        images = images.cuda(non_blocking=True)
        labels = torch.tensor(labels, dtype=torch.long).cuda(non_blocking=True)
    else:
        labels = torch.tensor(labels, dtype=torch.long)

    optimizer.zero_grad() # zero the gradient buff
    output = model(images)

    loss = F.nll_loss(output, labels)
    if use_cuda:
        loss = loss.cuda()

    loss.backward() # retain_graph=True
    optimizer.step()

    train_loss += loss.item() # Loss
    running_loss += loss.item()
    if batch_idx % 50 == 49:
        writer.add_scalar('training loss', running_loss/50, epoch * len(train_dataloader))
        running_loss = 0
    #print('##### Epoch:', epoch, ', -- batch:', batch_idx, '\n')
    #print('Train Loss: %.3f, accumulated average Loss: %.3f #####')
```

```
In [14]: def test(model, epoch, writer):
print('\n ##### Test phase, Epoch: {} #####')
model.eval()
with torch.no_grad():
    test_loss = 0
    correct = 0
    for (batch_idx, target_tuple) in enumerate(test_dataloader):
        images, labels = target_tuple
        if use_cuda:
            images = images.cuda(non_blocking=True)
            labels = torch.tensor(labels, dtype=torch.long).cuda(non_blocking=True)
        else:
            labels = torch.tensor(labels, dtype=torch.long)

        output = model(images)
        #print(output.shape)

        _, predicted = torch.max(output.data, 1)
```

```

        correct += (predicted == labels).sum().item()

        loss = F.nll_loss(output, labels)
        if use_cuda:
            loss = loss.cuda()

        test_loss += loss.item() # Loss
        #print('##### Epoch:', epoch, ', -- batch:', ba
        #      'Test Loss: %.3f, accumulated average Loss: %.3f #####'
acc = correct*100/len(test_data)
print('Accuracy: ', acc)
writer.add_scalar('test accuracy', acc, epoch)

```

```

In [15]: def train_and_test(model, optimizer, epoch_num = 5, summary_path='runs/mnist_experi
writer = SummaryWriter(summary_path)
for epoch in range(epoch_num):
    train(model, optimizer, epoch, writer)
    test(model, epoch, writer)

```

```

In [16]: train_and_test(model, optimizer, 5, 'runs/mnist_experiment_dropout=0')

```

```
##### Train phase, Epoch: 0 #####
Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 0 #####
Accuracy: 98.55

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 1 #####
Accuracy: 98.92

##### Train phase, Epoch: 2 #####
Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 2 #####
Accuracy: 98.95

##### Train phase, Epoch: 3 #####
Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 3 #####
Accuracy: 98.83

##### Train phase, Epoch: 4 #####
Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 4 #####
Accuracy: 98.9
```

In [18]: `%tensorboard --logdir=runs`

Reusing TensorBoard on port 6006 (pid 63044), started 0:27:38 ago. (Use '!kill 63044' to kill it.)



- Dropout rate = 0.5

```
In [19]: use_cuda = torch.cuda.is_available() #GPU cuda
best_loss = float('inf')

model = DeepModel(dropout_rate = 0.5)
```



```
optimizer = torch.optim.Adam(model.parameters())
if use_cuda:
    model = torch.nn.parallel.DataParallel(model.cuda()) # , device_ids=[0, 1, 2,
    torch.backends.cudnn.benchmark = True
```

```
In [20]: train_and_test(model, optimizer, 5, 'runs/mnist_experiment_dropout=0.5')
```

```
##### Train phase, Epoch: 0 #####
```

```
Learning rate at this epoch is: 0.001
```

```
##### Test phase, Epoch: 0 #####
```

```
Accuracy: 98.68
```

```
##### Train phase, Epoch: 1 #####
```

```
Learning rate at this epoch is: 0.001
```

```
##### Test phase, Epoch: 1 #####
```

```
Accuracy: 99.04
```

```
##### Train phase, Epoch: 2 #####
```

```
Learning rate at this epoch is: 0.001
```

```
##### Test phase, Epoch: 2 #####
```

```
Accuracy: 99.03
```

```
##### Train phase, Epoch: 3 #####
```

```
Learning rate at this epoch is: 0.001
```

KeyboardInterrupt

Traceback (most recent call last)

Cell In[20], line 1

```
----> 1 train_and_test(model, optimizer, 5, )
```

Cell In[15], line 4, in train_and_test(model, optimizer, epoch_num, summary_path)

```
2 writer = SummaryWriter(summary_path)
3 for epoch in range(epoch_num):
----> 4     train(model, optimizer, epoch, writer)
5     test(model, epoch, writer)
```

Cell In[13], line 22, in train(model, optimizer, epoch, writer)

```
19 if use_cuda:
20     loss = loss.cuda()
---> 22 loss.backward() # retain_graph=True
23 optimizer.step()
25 train_loss += loss.item() # loss
```

File ~\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\torch\tensor.py:625, in Tensor.backward(self, gradient, retain_graph, create_graph, inputs)

```
615 if has_torch_function_unary(self):
616     return handle_torch_function(
617         Tensor.backward,
618         (self,),
619         (...), 623         inputs=inputs,
624         )
--> 625 torch.autograd.backward(
626     self, gradient, retain_graph, create_graph, inputs=inputs
627 )
```

File ~\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\torch\autograd__init__.py:354, in backward(tensors, grad_tensors, retain_graph, create_graph, grad_variables, inputs)

```
349 retain_graph = create_graph
351 # The reason we repeat the same comment below is that
352 # some Python versions print out the first line of a multi-line function
353 # calls in the traceback and some print out the last line
--> 354 _engine_run_backward(
355     tensors,
356     grad_tensors_,
357     retain_graph,
358     create_graph,
359     inputs_tuple,
360     allow_unreachable=True,
361     accumulate_grad=True,
362 )
```

File ~\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\torch\autograd\graph.py:841, in _engine_run_backward(t_outputs, *args, **kwargs)

```
839 unregister_hooks = _register_logging_hooks_on_whole_graph(t_outputs)
840 try:
--> 841     return Variable._execution_engine.run_backward( # Calls into the C++ engine to run the backward pass
842         t_outputs, *args, **kwargs
```

```
843     ) # Calls into the C++ engine to run the backward pass
844 finally:
845     if attach_logging_hooks:
```

KeyboardInterrupt:

In [21]: `%tensorboard --logdir=runs`

Reusing TensorBoard on port 6006 (pid 63044), started 0:32:31 ago. (Use '!kill 63044' to kill it.)



Phần 2.2: Vai trò của activation function

```
In [23]: class SimpleModel(nn.Module):  
    def __init__(self, dropout_rate = 0.5, activation = None):  
        super(SimpleModel, self).__init__()  
        self.dense_1 = nn.Linear(28*28, 512)  
        self.dropout = nn.Dropout(dropout_rate)
```

```

        self.dense_2 = nn.Linear(512, 10)
        self.activation = activation

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.dense_1(x)
        if self.activation is not None:
            x = self.activation(x)
        x = self.dropout(x)
        x = self.dense_2(x)
        output = F.log_softmax(x)

    return output

```

Kiểm tra số lượng tham số

```

In [24]: simple_model = SimpleModel()
if use_cuda:
    simple_model = simple_model.cuda()
for param in simple_model.parameters():
    if param.requires_grad:
        print('param autograd')
        break

input = torch.rand(1, 28, 28)
if use_cuda:
    input = input.cuda()
output = simple_model(input) # type: torch.Tensor

model_parameters = filter(lambda p: p.requires_grad, simple_model.parameters())
params = sum([np.prod(p.size()) for p in model_parameters])
print('Number of parameter:', params)

```

param autograd

Number of parameter: 407050

Trước tiên hãy thử xem, sẽ thế nào nếu không sử dụng hàm kích hoạt cho lớp ẩn:

```

In [28]: # YOUR CODE HERE
simple_model = SimpleModel(dropout_rate = 0.5, activation = None)
if use_cuda:
    simple_model = simple_model.cuda()
optimizer = torch.optim.Adam(simple_model.parameters())
# YOUR CODE HERE
train_and_test(simple_model, optimizer, 5, 'runs/mnist_none_activation')

```

```
##### Train phase, Epoch: 0 #####
Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 0 #####
Accuracy: 90.27

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 1 #####
Accuracy: 91.84

##### Train phase, Epoch: 2 #####
Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 2 #####
Accuracy: 91.72

##### Train phase, Epoch: 3 #####
Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 3 #####
Accuracy: 91.18

##### Train phase, Epoch: 4 #####
Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 4 #####
Accuracy: 91.91
```

Kết quả đạt được là tương đối tệ trên tập dữ liệu MNIST. Tiếp theo chúng ta sẽ thử nghiệm và so sánh kết quả khi sử dụng các hàm kích hoạt khác nhau:

```
In [30]: # YOUR CODE HERE
for activation in [None, nn.Sigmoid(), nn.Tanh(), nn.ReLU()]:
    print('Activation: ', str(activation))
    simple_model = SimpleModel(dropout_rate = 0.5, activation = activation)
    if use_cuda:
        simple_model = simple_model.cuda()
    optimizer = torch.optim.Adam(simple_model.parameters())
    # YOUR CODE HERE
    train_and_test(simple_model, optimizer, 5, 'runs/mnist_' + str(activation))
```

Activation: None

Train phase, Epoch: 0

Learning rate at this epoch is: 0.001

Test phase, Epoch: 0

Accuracy: 91.54

Train phase, Epoch: 1

Learning rate at this epoch is: 0.001

Test phase, Epoch: 1

Accuracy: 91.51

Train phase, Epoch: 2

Learning rate at this epoch is: 0.001

Test phase, Epoch: 2

Accuracy: 92.16

Train phase, Epoch: 3

Learning rate at this epoch is: 0.001

Test phase, Epoch: 3

Accuracy: 91.76

Train phase, Epoch: 4

Learning rate at this epoch is: 0.001

Test phase, Epoch: 4

Accuracy: 91.34

Activation: Sigmoid()

Train phase, Epoch: 0

Learning rate at this epoch is: 0.001

Test phase, Epoch: 0

Accuracy: 93.59

Train phase, Epoch: 1

Learning rate at this epoch is: 0.001

```
##### Test phase, Epoch: 1 #####
Accuracy: 95.52

##### Train phase, Epoch: 2 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 2 #####
Accuracy: 96.5

##### Train phase, Epoch: 3 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 3 #####
Accuracy: 97.03

##### Train phase, Epoch: 4 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 4 #####
Accuracy: 97.27
Activation: Tanh()

##### Train phase, Epoch: 0 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 0 #####
Accuracy: 94.28

##### Train phase, Epoch: 1 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 1 #####
Accuracy: 96.02

##### Train phase, Epoch: 2 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 2 #####
Accuracy: 96.65

##### Train phase, Epoch: 3 #####

Learning rate at this epoch is: 0.001
```



```

##### Test phase, Epoch: 3 #####
Accuracy: 96.81

##### Train phase, Epoch: 4 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 4 #####
Accuracy: 97.02
Activation: ReLU()

##### Train phase, Epoch: 0 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 0 #####
Accuracy: 96.39

##### Train phase, Epoch: 1 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 1 #####
Accuracy: 97.2

##### Train phase, Epoch: 2 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 2 #####
Accuracy: 97.5

##### Train phase, Epoch: 3 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 3 #####
Accuracy: 97.79

##### Train phase, Epoch: 4 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 4 #####
Accuracy: 97.91

```

In [31]: `%tensorboard --logdir=runs`

Reusing TensorBoard on port 6006 (pid 63044), started 0:13:21 ago. (Use '!kill 63044' to kill it.)



Phần 2.3: Vai trò của hệ số học learning rate

Ta tiếp tục quan sát tác động của learning rate đến quá trình học của mạng:

```
In [33]: # YOUR CODE HERE
learning_rates = [1E-0, 1E-1, 1E-2, 1E-3, 1E-4, 1E-5, 1E-6, 1E-7]
for lr in learning_rates:
    # YOUR CODE HERE
    print('Learning rate = %f' % lr)
    simple_model = SimpleModel(dropout_rate = 0.5, activation = nn.ReLU())
    if use_cuda:
        simple_model = simple_model.cuda()
    optimizer = torch.optim.SGD(simple_model.parameters(), lr = lr, momentum = 0.9)
    # YOUR CODE HERE
    train_and_test(simple_model, optimizer, 5, 'runs/lr_' + str(lr))
# YOUR CODE HERE
```

Learning rate = 1.000000

Train phase, Epoch: 0

Learning rate at this epoch is: 1.0

Test phase, Epoch: 0 #####
Accuracy: 10.33

Train phase, Epoch: 1

Learning rate at this epoch is: 1.0

Test phase, Epoch: 1 #####
Accuracy: 11.36

Train phase, Epoch: 2

Learning rate at this epoch is: 1.0

Test phase, Epoch: 2 #####
Accuracy: 11.36

Train phase, Epoch: 3

Learning rate at this epoch is: 1.0

Test phase, Epoch: 3 #####
Accuracy: 10.29

Train phase, Epoch: 4

Learning rate at this epoch is: 1.0

Test phase, Epoch: 4 #####
Accuracy: 11.36
Learning rate = 0.100000

Train phase, Epoch: 0

Learning rate at this epoch is: 0.1

Test phase, Epoch: 0 #####
Accuracy: 91.11

Train phase, Epoch: 1

Learning rate at this epoch is: 0.1

```
##### Test phase, Epoch: 1 #####
Accuracy: 93.15

##### Train phase, Epoch: 2 #####

Learning rate at this epoch is: 0.1

##### Test phase, Epoch: 2 #####
Accuracy: 92.69

##### Train phase, Epoch: 3 #####

Learning rate at this epoch is: 0.1

##### Test phase, Epoch: 3 #####
Accuracy: 93.35

##### Train phase, Epoch: 4 #####

Learning rate at this epoch is: 0.1

##### Test phase, Epoch: 4 #####
Accuracy: 94.16
Learning rate = 0.010000

##### Train phase, Epoch: 0 #####

Learning rate at this epoch is: 0.01

##### Test phase, Epoch: 0 #####
Accuracy: 95.15

##### Train phase, Epoch: 1 #####

Learning rate at this epoch is: 0.01

##### Test phase, Epoch: 1 #####
Accuracy: 96.55

##### Train phase, Epoch: 2 #####

Learning rate at this epoch is: 0.01

##### Test phase, Epoch: 2 #####
Accuracy: 97.17

##### Train phase, Epoch: 3 #####

Learning rate at this epoch is: 0.01
```

```
##### Test phase, Epoch: 3 #####
Accuracy: 97.35

##### Train phase, Epoch: 4 #####

Learning rate at this epoch is: 0.01

##### Test phase, Epoch: 4 #####
Accuracy: 97.7
Learning rate = 0.001000

##### Train phase, Epoch: 0 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 0 #####
Accuracy: 89.36

##### Train phase, Epoch: 1 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 1 #####
Accuracy: 91.22

##### Train phase, Epoch: 2 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 2 #####
Accuracy: 92.36

##### Train phase, Epoch: 3 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 3 #####
Accuracy: 93.17

##### Train phase, Epoch: 4 #####

Learning rate at this epoch is: 0.001

##### Test phase, Epoch: 4 #####
Accuracy: 93.68
Learning rate = 0.000100

##### Train phase, Epoch: 0 #####
```

Learning rate at this epoch is: 0.0001

Test phase, Epoch: 0 #####
Accuracy: 72.9

Train phase, Epoch: 1

Learning rate at this epoch is: 0.0001

Test phase, Epoch: 1 #####
Accuracy: 79.83

Train phase, Epoch: 2

Learning rate at this epoch is: 0.0001

Test phase, Epoch: 2 #####
Accuracy: 83.5

Train phase, Epoch: 3

Learning rate at this epoch is: 0.0001

Test phase, Epoch: 3 #####
Accuracy: 85.3

Train phase, Epoch: 4

Learning rate at this epoch is: 0.0001

Test phase, Epoch: 4 #####
Accuracy: 86.64
Learning rate = 0.00010

Train phase, Epoch: 0

Learning rate at this epoch is: 1e-05

Test phase, Epoch: 0 #####
Accuracy: 22.4

Train phase, Epoch: 1

Learning rate at this epoch is: 1e-05

Test phase, Epoch: 1 #####
Accuracy: 39.72

Train phase, Epoch: 2

Learning rate at this epoch is: 1e-05

Test phase, Epoch: 2 #####
Accuracy: 52.17

Train phase, Epoch: 3

Learning rate at this epoch is: 1e-05

Test phase, Epoch: 3 #####
Accuracy: 60.31

Train phase, Epoch: 4

Learning rate at this epoch is: 1e-05

Test phase, Epoch: 4 #####
Accuracy: 64.88
Learning rate = 0.000001

Train phase, Epoch: 0

Learning rate at this epoch is: 1e-06

Test phase, Epoch: 0 #####
Accuracy: 10.62

Train phase, Epoch: 1

Learning rate at this epoch is: 1e-06

Test phase, Epoch: 1 #####
Accuracy: 11.5

Train phase, Epoch: 2

Learning rate at this epoch is: 1e-06

Test phase, Epoch: 2 #####
Accuracy: 12.3

Train phase, Epoch: 3

Learning rate at this epoch is: 1e-06

Test phase, Epoch: 3 #####
Accuracy: 13.16


```

##### Train phase, Epoch: 4 #####
Learning rate at this epoch is: 1e-06

##### Test phase, Epoch: 4 #####
Accuracy: 14.09
Learning rate = 0.000000

##### Train phase, Epoch: 0 #####
Learning rate at this epoch is: 1e-07

##### Test phase, Epoch: 0 #####
Accuracy: 11.22

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: 1e-07

##### Test phase, Epoch: 1 #####
Accuracy: 11.33

##### Train phase, Epoch: 2 #####
Learning rate at this epoch is: 1e-07

##### Test phase, Epoch: 2 #####
Accuracy: 11.43

##### Train phase, Epoch: 3 #####
Learning rate at this epoch is: 1e-07

##### Test phase, Epoch: 3 #####
Accuracy: 11.53

##### Train phase, Epoch: 4 #####
Learning rate at this epoch is: 1e-07

##### Test phase, Epoch: 4 #####
Accuracy: 11.69

```

In [34]: `%tensorboard --logdir=runs`

Reusing TensorBoard on port 6006 (pid 63044), started 0:19:02 ago. (Use '!kill 63044' to kill it.)



- Giảm learning rate theo cơ chế: $\text{lnr} = \text{init_lnr} / (1 + \text{decay} * \text{step})$

```
In [27]: def train(model, optimizer, scheduler, epoch, writer):  
          print('\n ##### Train phase, Epoch: {} #####'  
          model.train()  
          train_loss = 0
```

```

running_loss = 0
print('\nLearning rate at this epoch is: ', scheduler.get_last_lr(), '\n')
for (batch_idx, target_tuple) in enumerate(train_dataloader):
    images, labels = target_tuple
    if use_cuda:
        images = images.cuda(non_blocking=True)
        labels = torch.tensor(labels, dtype=torch.long).cuda(non_blocking=True)
    else:
        labels = torch.tensor(labels, dtype=torch.long)

    optimizer.zero_grad() # zero the gradient buff
    output = model(images)

    loss = F.nll_loss(output, labels)
    if use_cuda:
        loss = loss.cuda()

    loss.backward() # retain_graph=True
    optimizer.step()

    train_loss += loss.item() # Loss
    running_loss += loss.item()
    if batch_idx % 50 == 49:
        writer.add_scalar('training loss', running_loss/50, epoch * len(train_dataloader))
        running_loss = 0
scheduler.step()

```

```

In [28]: def train_and_test(model, optimizer, scheduler, epoch_num = 5, summary_path='runs/mnist'):
writer = SummaryWriter(summary_path)
for epoch in range(epoch_num):
    train(model, optimizer, scheduler, epoch, writer)
    test(model, epoch, writer)

```

```

In [29]: if 'SimpleModel' not in globals():
class SimpleModel(nn.Module):
    def __init__(self, dropout_rate=0.5, activation=None):
        super(SimpleModel, self).__init__()
        self.dense_1 = nn.Linear(28*28, 512)
        self.dropout = nn.Dropout(dropout_rate)
        self.dense_2 = nn.Linear(512, 10)
        self.activation = activation

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = self.dense_1(x)
        if self.activation is not None:
            x = self.activation(x)
        x = self.dropout(x)
        x = self.dense_2(x)
        return F.log_softmax(x, dim=1)

simple_model = SimpleModel(dropout_rate=0.5, activation=nn.ReLU())
if use_cuda:
    simple_model = simple_model.cuda()
optimizer = torch.optim.SGD(simple_model.parameters(), lr=0.01, momentum=0.9)

```

```
scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.9)
train_and_test(simple_model, optimizer, scheduler, 20, 'runs/exponentialLR')
if use_cuda:
    simple_model = simple_model.cuda()
optimizer = torch.optim.SGD(simple_model.parameters(), lr = 0.01, momentum = 0.9)
scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.9)
# YOUR CODE HERE
train_and_test(simple_model, optimizer, scheduler, 20, 'runs/exponentialLR')
```

```
##### Train phase, Epoch: 0 #####
Learning rate at this epoch is: [0.01]

##### Test phase, Epoch: 0 #####
Accuracy: 95.11

##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: [0.009000000000000001]

##### Test phase, Epoch: 1 #####
Accuracy: 96.4

##### Train phase, Epoch: 2 #####
Learning rate at this epoch is: [0.008100000000000001]

##### Test phase, Epoch: 2 #####
Accuracy: 96.93

##### Train phase, Epoch: 3 #####
Learning rate at this epoch is: [0.007290000000000001]

##### Test phase, Epoch: 3 #####
Accuracy: 97.38

##### Train phase, Epoch: 4 #####
Learning rate at this epoch is: [0.006561000000000002]

##### Test phase, Epoch: 4 #####
Accuracy: 97.54

##### Train phase, Epoch: 5 #####
Learning rate at this epoch is: [0.005904900000000002]

##### Test phase, Epoch: 5 #####
Accuracy: 97.77

##### Train phase, Epoch: 6 #####
Learning rate at this epoch is: [0.005314410000000002]

##### Test phase, Epoch: 6 #####
Accuracy: 97.85
```

```
##### Train phase, Epoch: 7 #####
Learning rate at this epoch is: [0.00478296900000002]

##### Test phase, Epoch: 7 #####
Accuracy: 97.93

##### Train phase, Epoch: 8 #####
Learning rate at this epoch is: [0.004304672100000002]

##### Test phase, Epoch: 8 #####
Accuracy: 97.99

##### Train phase, Epoch: 9 #####
Learning rate at this epoch is: [0.003874204890000002]

##### Test phase, Epoch: 9 #####
Accuracy: 97.96

##### Train phase, Epoch: 10 #####
Learning rate at this epoch is: [0.003486784401000002]

##### Test phase, Epoch: 10 #####
Accuracy: 98.08

##### Train phase, Epoch: 11 #####
Learning rate at this epoch is: [0.003138105960900002]

##### Test phase, Epoch: 11 #####
Accuracy: 98.04

##### Train phase, Epoch: 12 #####
Learning rate at this epoch is: [0.0028242953648100018]

##### Test phase, Epoch: 12 #####
Accuracy: 98.07

##### Train phase, Epoch: 13 #####
Learning rate at this epoch is: [0.0025418658283290017]

##### Test phase, Epoch: 13 #####
Accuracy: 98.15
```

```
##### Train phase, Epoch: 14 #####
Learning rate at this epoch is: [0.0022876792454961017]

##### Test phase, Epoch: 14 #####
Accuracy: 98.11

##### Train phase, Epoch: 15 #####
Learning rate at this epoch is: [0.0020589113209464917]

##### Test phase, Epoch: 15 #####
Accuracy: 98.17

##### Train phase, Epoch: 16 #####
Learning rate at this epoch is: [0.0018530201888518425]

##### Test phase, Epoch: 16 #####
Accuracy: 98.17

##### Train phase, Epoch: 17 #####
Learning rate at this epoch is: [0.0016677181699666583]

##### Test phase, Epoch: 17 #####
Accuracy: 98.18

##### Train phase, Epoch: 18 #####
Learning rate at this epoch is: [0.0015009463529699924]

##### Test phase, Epoch: 18 #####
Accuracy: 98.22

##### Train phase, Epoch: 19 #####
Learning rate at this epoch is: [0.0013508517176729932]

##### Test phase, Epoch: 19 #####
Accuracy: 98.27

##### Train phase, Epoch: 0 #####
Learning rate at this epoch is: [0.01]

##### Test phase, Epoch: 0 #####
Accuracy: 98.13
```

```
##### Train phase, Epoch: 1 #####
Learning rate at this epoch is: [0.009000000000000001]

##### Test phase, Epoch: 1 #####
Accuracy: 98.23

##### Train phase, Epoch: 2 #####
Learning rate at this epoch is: [0.008100000000000001]

##### Test phase, Epoch: 2 #####
Accuracy: 98.26

##### Train phase, Epoch: 3 #####
Learning rate at this epoch is: [0.007290000000000001]

##### Test phase, Epoch: 3 #####
Accuracy: 98.29

##### Train phase, Epoch: 4 #####
Learning rate at this epoch is: [0.006561000000000002]

##### Test phase, Epoch: 4 #####
Accuracy: 98.21

##### Train phase, Epoch: 5 #####
Learning rate at this epoch is: [0.005904900000000002]

##### Test phase, Epoch: 5 #####
Accuracy: 98.3

##### Train phase, Epoch: 6 #####
Learning rate at this epoch is: [0.005314410000000002]

##### Test phase, Epoch: 6 #####
Accuracy: 98.29

##### Train phase, Epoch: 7 #####
Learning rate at this epoch is: [0.004782969000000002]

##### Test phase, Epoch: 7 #####
Accuracy: 98.36
```



```
##### Train phase, Epoch: 8 #####
Learning rate at this epoch is: [0.004304672100000002]

##### Test phase, Epoch: 8 #####
Accuracy: 98.37

##### Train phase, Epoch: 9 #####
Learning rate at this epoch is: [0.003874204890000002]

##### Test phase, Epoch: 9 #####
Accuracy: 98.32

##### Train phase, Epoch: 10 #####
Learning rate at this epoch is: [0.003486784401000002]

##### Test phase, Epoch: 10 #####
Accuracy: 98.29

##### Train phase, Epoch: 11 #####
Learning rate at this epoch is: [0.003138105960900002]

##### Test phase, Epoch: 11 #####
Accuracy: 98.33

##### Train phase, Epoch: 12 #####
Learning rate at this epoch is: [0.0028242953648100018]

##### Test phase, Epoch: 12 #####
Accuracy: 98.43

##### Train phase, Epoch: 13 #####
Learning rate at this epoch is: [0.0025418658283290017]

##### Test phase, Epoch: 13 #####
Accuracy: 98.33

##### Train phase, Epoch: 14 #####
Learning rate at this epoch is: [0.0022876792454961017]

##### Test phase, Epoch: 14 #####
Accuracy: 98.39
```

```
##### Train phase, Epoch: 15 #####
Learning rate at this epoch is: [0.0020589113209464917]

##### Test phase, Epoch: 15 #####
Accuracy: 98.38

##### Train phase, Epoch: 16 #####
Learning rate at this epoch is: [0.0018530201888518425]

##### Test phase, Epoch: 16 #####
Accuracy: 98.32

##### Train phase, Epoch: 17 #####
Learning rate at this epoch is: [0.0016677181699666583]

##### Test phase, Epoch: 17 #####
Accuracy: 98.4

##### Train phase, Epoch: 18 #####
Learning rate at this epoch is: [0.0015009463529699924]

##### Test phase, Epoch: 18 #####
Accuracy: 98.39

##### Train phase, Epoch: 19 #####
Learning rate at this epoch is: [0.0013508517176729932]

##### Test phase, Epoch: 19 #####
Accuracy: 98.38
```

```
In [30]: %tensorboard --logdir=runs
```

```
Reusing TensorBoard on port 6006 (pid 63044), started 0:38:38 ago. (Use '!kill 63044' to kill it.)
```

